

Artificial Intelligence Project 2: Self-Supervised Learning

Chih-Heng Hsiao

Student ID: 413551034

<https://github.com/CH-Joshua-Hsiao/NYCU-AI-HW2>

1 Introduction and Motivation

In this project, we implement SimCLR, a popular Self-Supervised Learning (SSL) method. Usually, training deep learning models requires a lot of labeled data, which is hard to get. SimCLR tries to solve this by learning features from unlabeled images.

Our main goals are to answer these questions:

1. How close can an SSL model get to a fully supervised model on CIFAR-10?
2. How well do the features learned by SSL transfer to a different dataset like CIFAR-100?
3. How do hyperparameters like temperature, batch size, and the projector head affect the training?

2 Methods

The project was built using PyTorch. *Note: AI tools like Gemini were used to help with code structure and LaTeX formatting.*

2.1 Dataset and Data Augmentation

We used the CIFAR-10 dataset (32×32 RGB images, 10 classes). In SimCLR, data augmentation is very important. We apply two random augmentations to each image to create a positive pair. Our augmentation pipeline includes:

- **Random Resized Cropping:** Helps the model recognize objects from different sizes and parts.
- **Random Horizontal Flipping:** Helps with spatial invariance.
- **Color Jittering (80%) & Grayscale (20%):** This is a critical step. If we don't use color augmentations, the network might just look at the color distribution as a "shortcut" to match the images, instead of learning the actual shape of the objects.

2.2 Model Architecture

We modified the standard ResNet-18 to work with the small 32×32 CIFAR images. We changed the first 7×7 conv layer to a 3×3 conv layer and removed the first max pooling layer. This prevents the image from shrinking too fast. The final classification layer was replaced with a global average pooling layer, giving a 512-dimensional output vector h .

2.3 Projector Head

We added a 2-layer MLP (linear $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ linear) on top of the backbone. This head maps the 512-dimensional vector h into a smaller 128-dimensional vector z .

2.4 Loss Function

We used the NT-Xent (Normalized Temperature-scaled Cross Entropy) loss. It calculates the cosine similarity between the positive pair and scales it by a temperature parameter ($\tau = 0.5$). It tries to pull the positive pair together while pushing all other negative samples in the batch away.

3 Experiments and Results

We trained the SimCLR baseline for 200 epochs with a batch size of 256. For evaluation (Linear Probing), we froze the backbone and trained a simple linear classifier for 100 epochs.

3.1 Baseline Comparisons

To see how good the learned features are, we compared three models: 1. **Random Baseline**: A frozen ResNet-18 with random weights. 2. **SimCLR**: Our trained SSL model (frozen) with a linear classifier. 3. **Supervised Learning (SL)**: A ResNet-18 trained from scratch with labels.

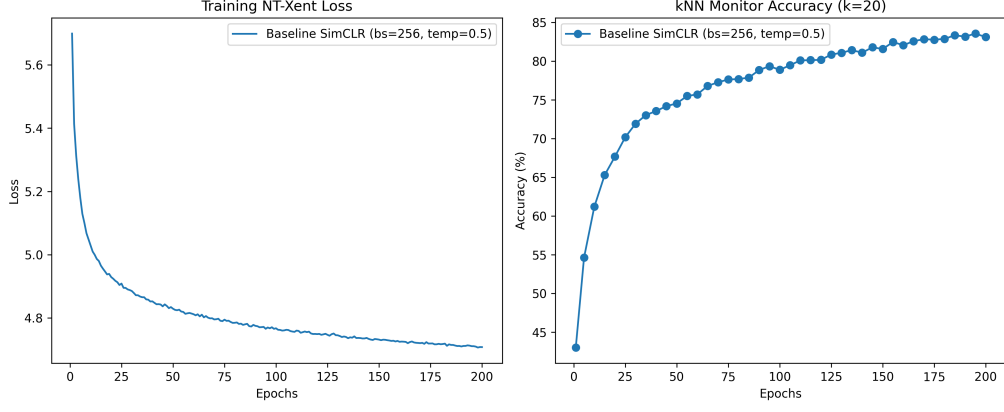


Figure 1: SimCLR training loss and K-NN accuracy ($k = 200$) over 200 epochs.

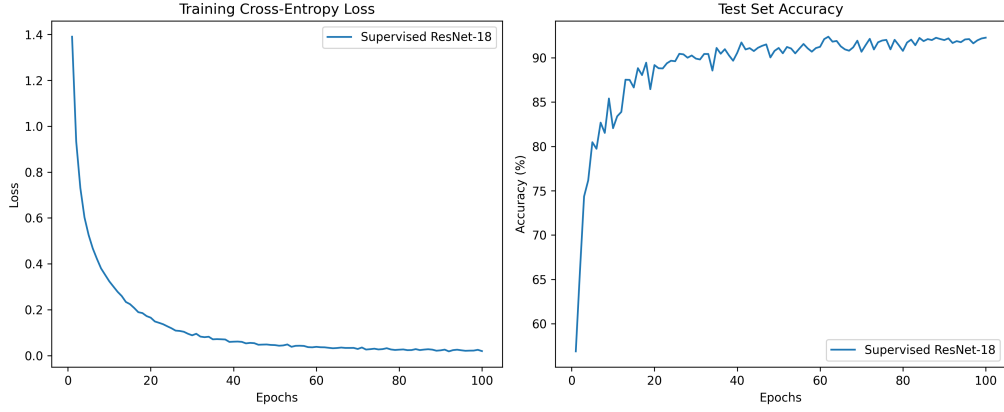


Figure 2: Supervised Learning training curves.

Model	CIFAR-10 Test Accuracy (%)
Random Initialization Backbone	33.79%
SimCLR SSL + Linear Probing	82.82%
Supervised Learning	92.26%

Table 1: Accuracy comparison on CIFAR-10.

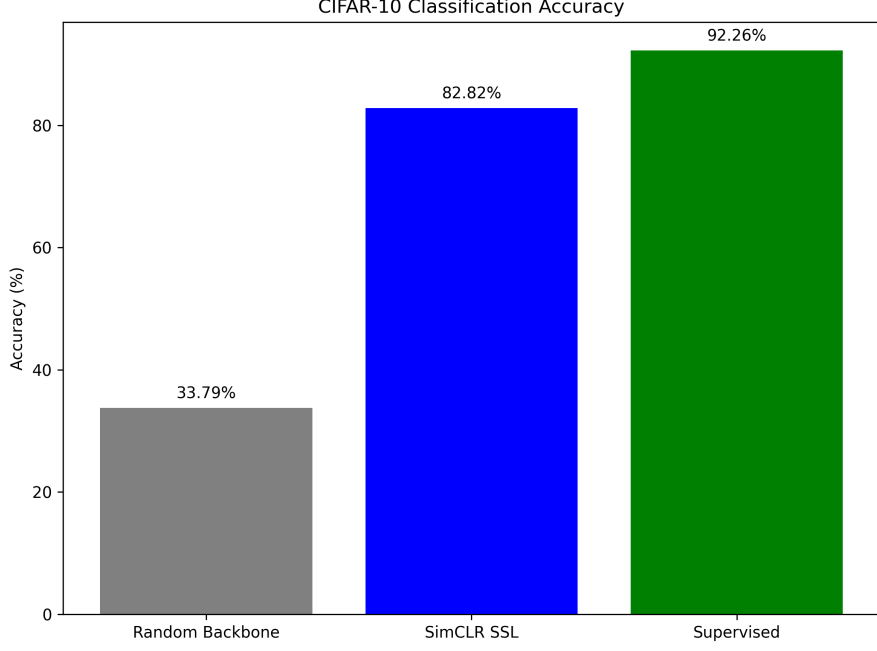


Figure 3: Comparison between random, SSL, and Supervised Learning.

The Random Initialization backbone gets 33.79%, which is better than random guessing (10%) because random high-dimensional projections still keep some basic image structure. However, SimCLR improves this significantly to 82.82% without ever seeing any labels during training.

3.2 Ablation Studies

1. Temperature (τ):

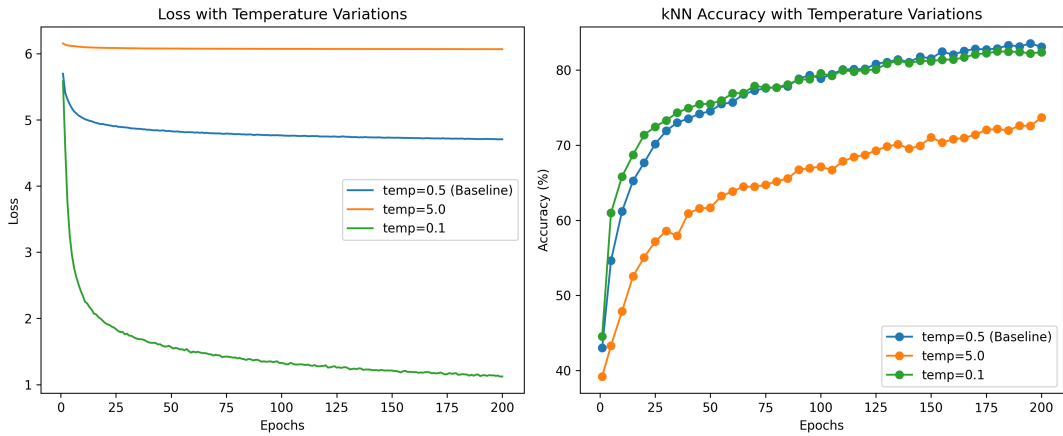


Figure 4: Training with different temperature values.

Are the changes of the two learning curves consistent?

No. When using $\tau = 0.1$, the loss drops very low, but the final kNN accuracy (82.39%) is actually worse than the baseline $\tau = 0.5$ (83.10%). A small temperature makes the model

overconfident on a few easy negative samples, so the loss looks good but the features are not actually better. High temperature ($\tau = 5.0$) makes everything too uniform, so both loss and accuracy get worse.

2. Batch Size:

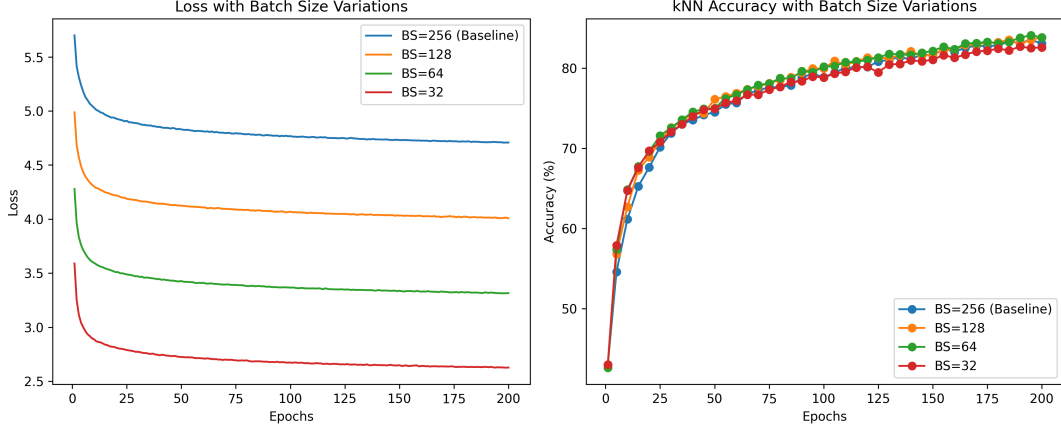


Figure 5: Training with different batch sizes.

Surprisingly, the accuracy stayed very stable (around 82-83%) even when we dropped the batch size to 32. This is likely because CIFAR-10 is a simple dataset with only 10 classes. Even a small batch size provides enough different negative samples for the model to learn good features.

3. Projector Head:

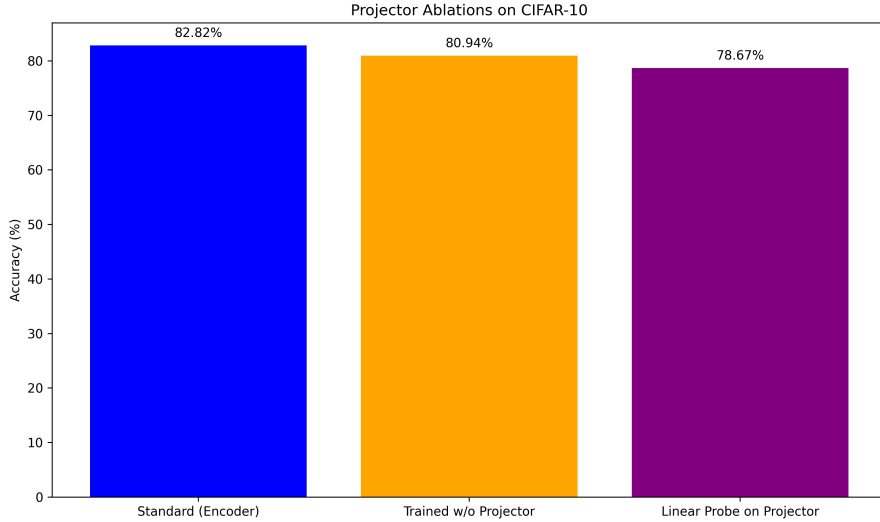


Figure 6: Testing the impact of the projector head.

If we train without the projector (calculate loss directly on the 512-dim output), accuracy drops to 80.94%. If we train with the projector but evaluate on its 128-dim output instead of the 512-dim output, accuracy drops to 78.67%. This shows the projector acts as an "augmentation absorber." It throws away details to match the augmented images, leaving the useful features safe inside the 512-dim encoder output.

3.3 Transfer Learning (CIFAR-100)

To see how well the models generalize, we took the models trained on CIFAR-10 and evaluated them on CIFAR-100 (which has 100 new classes).

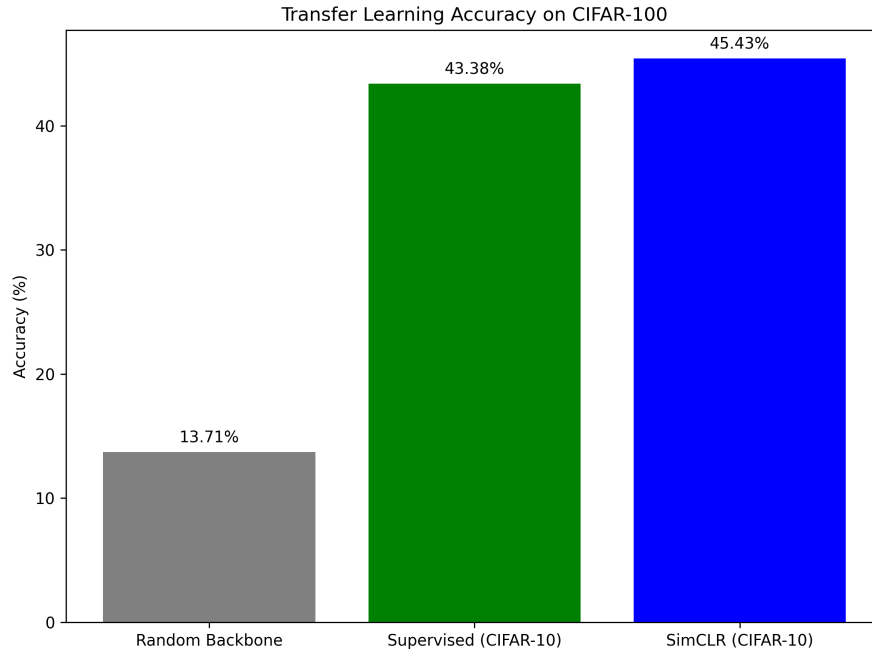


Figure 7: Transfer Learning results on CIFAR-100.

Frozen Backbone	CIFAR-100 Transfer Accuracy (%)
Random Initialization	13.71%
Supervised Learning	43.38%
SimCLR SSL	45.43%

Table 2: Transfer Learning accuracy comparison.

SimCLR actually beat the Supervised model here (45.43% vs 43.38%). This shows that SSL learns more general features, while SL overfits to the original 10 classes.

4 Discussion

4.1 Are the results and observed behaviors what you expect?

My initial expectation, based on the theoretical concepts of contrastive learning, was that SimCLR would be able to learn strong, useful features entirely without labels. That proved true, as my model reached an impressive 82.82% accuracy just by looking at augmented images. However, there were a few things that really surprised me during the experiments.

First, when I compared the Supervised Learning (SL) model with SSL, I expected SSL to be close. But seeing SL hit 92.26% on CIFAR-10 made me realize just how incredibly powerful direct label supervision is when testing on the exact same dataset. Second, the kNN monitor we implemented was a lifesaver. I was watching the training logs, and sometimes the contrastive loss would jump around or look weird, but the kNN accuracy kept steadily climbing. It was really cool to see the representation quality improve in real-time.

But the biggest surprise was the batch size experiment. All the literature and papers say SimCLR needs massive batch sizes (like 4096 or 8192) to get enough negative samples. When I ran the code with a batch size of 32, I honestly expected the model to fail completely and learn nothing. I was genuinely surprised to see that dropping the batch size down to 32 barely hurt the performance at all (still getting around 82%). After thinking about it, I realized this makes perfect sense: CIFAR-10 is a very simple dataset with only 10 classes. Even in a tiny batch of 32, the model is already seeing examples from almost all 10 categories, providing enough negative variety to learn good boundaries.

4.2 Factors affecting the results, including dataset characteristics

The most critical factor affecting my results, especially in the transfer learning phase, was the dataset itself. When I took the models trained on CIFAR-10 and tested them on CIFAR-100, the SL model's accuracy crashed to 43.38%, while the SSL model did slightly better at 45.43%.

This observation really made me understand the concept of "overfitting to a domain." The SL model basically memorized the 10 specific classes of CIFAR-10. For example, it might have learned to look at green backgrounds to classify frogs, completely ignoring the shape of the frog itself. When moved to CIFAR-100, those "shortcuts" became useless. SSL, on the other hand, didn't have any labels to rely on. It was forced to look at the actual shapes, edges, and textures of every single image to tell them apart from their augmented twins. This resulted in more robust, general features.

However, 45.43% is still a failing grade in terms of absolute accuracy. I believe the main limiting factor here is that CIFAR-10 lacks sheer visual diversity. It only has 50,000 tiny, blurry images across 10 basic categories. No matter how perfectly the contrastive loss is optimized, the model cannot magically learn semantic concepts or shapes it has never seen. To build a true "foundation model," SSL clearly needs to be trained on massive, diverse datasets like ImageNet.

4.3 Experiments I would do if there were more time available

If I had more time and didn't have to worry about GPU limits, I would love to deeply explore the data augmentations. I would run experiments where I completely turn off color jittering to see if the network really does "cheat" by using color histograms, as the

original paper warns. I would also like to try adding completely new augmentations, like Gaussian Blur or Random Rotation, to see if making the task harder helps or hurts the learning process.

I would also love to try training the SimCLR backbone for a much longer time, like 1000 or 2000 epochs. Supervised learning plateaus pretty quickly, but SSL methods are notoriously slow to converge. Since SSL doesn't have labels to overfit to, I am curious if SimCLR could eventually close the 10% gap with the SL model on CIFAR-10 if just given enough time to train. Finally, I would experiment with the projector head's output size (changing it from 128 to 64 or 256) to see exactly how much compression is ideal before doing the contrastive loss.

4.4 What I learned and remaining questions

Through this project, I learned how tricky it is to train self-supervised models compared to standard classification. Getting the hyperparameters right is critical. For example, I learned that raw training loss can be a terrible and deceptive metric in SSL. My temperature ablation ($\tau = 0.1$) showed a beautiful, low loss curve, but the actual kNN accuracy was worse than the baseline. This taught me the practical importance of constantly monitoring the representation space with a downstream metric (like kNN) instead of just blindly trusting the loss function. I also clearly saw how SL builds "brittle" task-specific features while SSL builds "general" features.

I have two main remaining questions. First, what is the theoretical limit of contrastive learning on small datasets? Even with infinite training time and perfect hyperparameter tuning, is it mathematically possible for an SSL model to completely match an SL model on CIFAR-10 without any human-provided labels? Second, I wonder how few labeled images we actually need to beat the SL model. It would be really interesting to take my pre-trained SSL backbone and fine-tune it with just 100 or 500 labeled images to see if it outperforms the model trained from scratch with 50,000 labels.

5 References

1. Ting Chen, Simon Kornblith, Mohammad Norouzi, Geoffrey Hinton (2020). A Simple Framework for Contrastive Learning of Visual Representations. *arXiv preprint arXiv:2002.05709*.

A Source Code

A.1 run_all.py

```
import subprocess
import sys

def run_cmd(cmd):
    print(f"\n===== RUNNING: {cmd} =====\n")
    result = subprocess.run(cmd, shell=True)
    if result.returncode != 0:
        print(f"Command failed with exit code {result.returncode}")
        sys.exit(result.returncode)

def main():
    # 1. Train Supervised Baseline (from scratch, 100 epochs)
    run_cmd("python train_supervised.py --epochs 100 --batch_size 256 --save_name sl_baseline.pth")

    # 2. Train SimCLR Baseline (using batch_size 256, 200 epochs)
    run_cmd("python train_simclr.py --epochs 200 --batch_size 256 --save_name simclr_baseline.pth")

    # 3. Linear Probing on SimCLR Baseline (100 epochs as requested in PDF)
    run_cmd("python train_linprobe.py --epochs 100 --pretrained simclr_baseline.pth --save_name linprobe_simclr.pth")

    # 4. Lower-Bound Random Baseline
    run_cmd("python train_linprobe.py --epochs 100 --random --save_name linprobe_random_baseline.pth")

    # 5. Temperature Ablations (200 epochs for fair comparison with baseline)
    run_cmd("python train_simclr.py --epochs 200 --batch_size 256 --temperature 5.0 --save_name simclr_temp5.pth")
    run_cmd("python train_simclr.py --epochs 200 --batch_size 256 --temperature 0.1 --save_name simclr_temp01.pth")

    # 6. Batch Size Ablations
    run_cmd("python train_simclr.py --epochs 200 --batch_size 128 --save_name simclr_bs128.pth")
    run_cmd("python train_simclr.py --epochs 200 --batch_size 64 --save_name simclr_bs64.pth")
    run_cmd("python train_simclr.py --epochs 200 --batch_size 32 --save_name simclr_bs32.pth")

    # 7. Projector Ablations
    # Train without projector
```

```

run_cmd("python train_simclr.py --epochs 200 --batch_size 256 --
        save_name simclr_noproj.pth --no_projector")
run_cmd("python train_linprobe.py --epochs 100 --pretrained
        simclr_noproj.pth --save_name linprobe_simclr_noproj.pth")
# Linprobe WITH projector outputs from baseline
run_cmd("python train_linprobe.py --epochs 100 --pretrained
        simclr_baseline.pth --save_name linprobe_simclr_projout.pth --
        use_projector")

# 8. Transfer Learning on CIFAR-100
run_cmd("python train_linprobe.py --epochs 100 --random --dataset
        cifar100 --save_name linprobe_random_cifar100.pth")
run_cmd("python train_linprobe.py --epochs 100 --pretrained_sl
        sl_baseline.pth --dataset cifar100 --save_name
        linprobe_sl_cifar100.pth")
run_cmd("python train_linprobe.py --epochs 100 --pretrained
        simclr_baseline.pth --dataset cifar100 --save_name
        linprobe_simclr_cifar100.pth")

# 9. Plot everything
run_cmd("python plot_results.py")
print("ALL EXPERIMENTS COMPLETED!")

if __name__ == "__main__":
    main()

```

A.2 dataset.py

```

import torchvision.transforms as transforms

class SimCLRTransform:
    def __init__(self, size=32):
        # Hyperparameters commonly used for CIFAR
        s = 1
        color_jitter = transforms.ColorJitter(0.8 * s, 0.8 * s, 0.8 * s
        , 0.2 * s)
        self.train_transform = transforms.Compose([
            transforms.RandomResizedCrop(size=size),
            transforms.RandomHorizontalFlip(),
            transforms.RandomApply([color_jitter], p=0.8),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            # CIFAR-10 stats
            transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023,
            0.1994, 0.2010))
        ])

    def __call__(self, x):
        return self.train_transform(x), self.train_transform(x)

```

```

def get_cifar10_transforms():
    test_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
            0.2010))
    ])
    train_sl_transform = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
            0.2010))
    ])
    return SimCLRTransform(size=32), train_sl_transform, test_transform

```

A.3 models.py

```

import torch
import torch.nn as nn
import torchvision.models as models

class ModifiedResNet18(nn.Module):
    def __init__(self):
        super().__init__()
        resnet = models.resnet18(weights=None)

        # Modify conv1: 3x3 kernel, stride=1, padding=1
        resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,
            padding=1, bias=False)
        # Modify maxpool to identity
        resnet.maxpool = nn.Identity()

        # Remove fc layer to get 512 dim output (before flatten, it's
            global avg pooling)
        self.encoder = nn.Sequential(*list(resnet.children())[:-1])

    def forward(self, x):
        h = self.encoder(x)
        h = h.view(h.shape[0], -1) # Flatten to (B, 512)
        return h

class ProjectorHead(nn.Module):
    def __init__(self, in_features=512, hidden_features=512,
        out_features=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, hidden_features),
            nn.BatchNorm1d(hidden_features),

```

```

        nn.ReLU(inplace=True),
        nn.Linear(hidden_features, out_features)
    )

    def forward(self, x):
        return self.net(x)

class SimCLRNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = ModifiedResNet18()
        self.projector = ProjectorHead()

    def forward(self, x):
        h = self.backbone(x)
        z = self.projector(h)
        return h, z

class LinearClassifier(nn.Module):
    def __init__(self, in_features=512, num_classes=10):
        super().__init__()
        self.fc = nn.Linear(in_features, num_classes)

    def forward(self, x):
        return self.fc(x)

```

A.4 loss.py

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class NTXentLoss(nn.Module):
    def __init__(self, temperature=0.5):
        super().__init__()
        self.temperature = temperature
        self.criterion = nn.CrossEntropyLoss(reduction="mean")

    def forward(self, z_i, z_j):
        """
        z_i and z_j are B x 128 normalized feature maps
        """
        batch_size = z_i.shape[0]
        device = z_i.device

        z_i = F.normalize(z_i, dim=1)
        z_j = F.normalize(z_j, dim=1)

        z = torch.cat([z_i, z_j], dim=0) # 2N x 128

```

```

# Pairwise cosine similarity matrix
sim_matrix = torch.matmul(z, z.T) # 2N x 2N
sim_matrix = sim_matrix / self.temperature

# Mask out self-similarity
mask = torch.eye(2 * batch_size, dtype=torch.bool).to(device)
sim_matrix.masked_fill_(mask, -9e15)

# Labels: the mate of i is i + N, mate of i + N is i
positives = torch.cat([torch.arange(batch_size, 2*batch_size),
    torch.arange(0, batch_size)]).to(device)

loss = self.criterion(sim_matrix, positives)
return loss

```

A.5 utils.py

```

import torch
import torch.nn.functional as F

def knn_predict(feature, feature_bank, feature_labels, classes, knn_k,
    knn_t):
    # compute cos similarity between each feature vector and feature
    # bank ---> [B, N]
    sim_matrix = torch.mm(feature, feature_bank)
    # [B, K]
    sim_weight, sim_indices = sim_matrix.topk(k=knn_k, dim=-1)
    # [B, K]
    sim_labels = torch.gather(feature_labels.expand(feature.size(0),
        -1), dim=-1, index=sim_indices)
    sim_weight = (sim_weight / knn_t).exp()

    # counts for each class
    one_hot_label = torch.zeros(feature.size(0) * knn_k, classes,
        device=sim_labels.device)
    # [B*K, C]
    one_hot_label = one_hot_label.scatter(dim=-1, index=sim_labels.view
        (-1, 1), value=1.0)
    # weighted score ---> [B, C]
    pred_scores = torch.sum(one_hot_label.view(feature.size(0), -1,
        classes) * sim_weight.unsqueeze(dim=-1), dim=1)

    pred_labels = pred_scores.argsort(dim=-1, descending=True)
    return pred_labels

def knn_monitor(net, memory_data_loader, test_data_loader, device, k
    =20, t=0.1, num_classes=10):

```

```

net.eval()
classes = num_classes
total_top1 = 0
total_num = 0
feature_bank = []

with torch.no_grad():
    # generate feature bank
    for data, target in memory_data_loader:
        # For SimCLR, memory loader returns (data, target)
        feature = net(data.to(device, non_blocking=True))
        feature = F.normalize(feature, dim=1)
        feature_bank.append(feature)

    # [D, N]
    feature_bank = torch.cat(feature_bank, dim=0).t().contiguous()

    # [N]
    feature_labels = torch.tensor(memory_data_loader.dataset.
        targets, device=device)

    # loop test data to predict the label by weighted knn search
    for data, target in test_data_loader:
        data, target = data.to(device, non_blocking=True), target.
            to(device, non_blocking=True)
        feature = net(data)
        feature = F.normalize(feature, dim=1)

        pred_labels = knn_predict(feature, feature_bank,
            feature_labels, classes, k, t)

        total_num += data.size(0)
        total_top1 += (pred_labels[:, 0] == target).float().sum().
            item()

return total_top1 / total_num * 100

```

A.6 train_simclr.py

```

import os
import tempfile
import torch
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision.datasets import CIFAR10
from tqdm import tqdm
import json

from dataset import get_cifar10_transforms

```

```

from models import SimCLRNet
from loss import NTXentLoss
from utils import knn_monitor

def train_simclr(batch_size=256, epochs=200, lr=3e-4, wd=1e-6,
    temperature=0.5, save_name="simclr_model.pth", no_projector=False):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    simclr_transform, _, test_transform = get_cifar10_transforms()

    train_dataset = CIFAR10(root='./data', train=True, transform=
        simclr_transform, download=True)
    train_loader = DataLoader(train_dataset, batch_size=batch_size,
        shuffle=True, num_workers=4, pin_memory=True, drop_last=True)

    memory_dataset = CIFAR10(root='./data', train=True, transform=
        test_transform, download=True)
    memory_loader = DataLoader(memory_dataset, batch_size=batch_size,
        shuffle=False, num_workers=4, pin_memory=True)

    test_dataset = CIFAR10(root='./data', train=False, transform=
        test_transform, download=True)
    test_loader = DataLoader(test_dataset, batch_size=batch_size,
        shuffle=False, num_workers=4, pin_memory=True)

    model = SimCLRNet().to(device)
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=wd)
    criterion = NTXentLoss(temperature=temperature)

    logs = {"loss": [], "knn_acc": []}

    for epoch in range(1, epochs + 1):
        model.train()
        total_loss = 0
        pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{epochs}")
        for (img1, img2), _ in pbar:
            img1, img2 = img1.to(device), img2.to(device)

            optimizer.zero_grad()
            h1, z1 = model(img1)
            h2, z2 = model(img2)

            if no_projector:
                loss = criterion(h1, h2)
            else:
                loss = criterion(z1, z2)
            loss.backward()
            optimizer.step()

```

```

        total_loss += loss.item()
        pbar.set_postfix({"Loss": loss.item()})

    avg_loss = total_loss / len(train_loader)
    logs["loss"].append(avg_loss)
    print(f"Epoch {epoch} | Avg Loss: {avg_loss:.4f}")

    if epoch % 5 == 0 or epoch == 1 or epoch == epochs:
        knn_acc = knn_monitor(model.backbone, memory_loader,
                               test_loader, device, k=20)
        logs["knn_acc"].append((epoch, knn_acc))
        print(f"Epoch {epoch} | kNN Monitor Accuracy: {knn_acc:.2f}
              }%")

    torch.save(model.state_dict(), save_name)
    with open(save_name.replace(".pth", "_logs.json"), "w") as f:
        json.dump(logs, f)

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--batch_size", type=int, default=256)
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--temperature", type=float, default=0.5)
    parser.add_argument("--save_name", type=str, default="
        simclr_baseline.pth")
    parser.add_argument("--no_projector", action="store_true")
    args = parser.parse_args()

    train_simclr(batch_size=args.batch_size, epochs=args.epochs,
                  temperature=args.temperature, save_name=args.save_name,
                  no_projector=args.no_projector)

```

A.7 train_linprobe.py

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision.datasets import CIFAR10, CIFAR100
from tqdm import tqdm
import json

from dataset import get_cifar10_transforms
from models import SimCLRNet, LinearClassifier, ModifiedResNet18

def train_linprobe(pretrained_path=None, pretrained_sl_path=None,
                   batch_size=256, epochs=100, lr=1e-3, wd=1e-6, save_name="
    linprobe_model.pth", is_random_baseline=False, dataset_name='cifar10

```



```

', use_projector=False):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    _, train_transform, test_transform = get_cifar10_transforms()

    if dataset_name == 'cifar100':
        train_dataset = CIFAR100(root='./data', train=True, transform=
            train_transform, download=True)
        test_dataset = CIFAR100(root='./data', train=False, transform=
            test_transform, download=True)
        num_classes = 100
    else:
        train_dataset = CIFAR10(root='./data', train=True, transform=
            train_transform, download=True)
        test_dataset = CIFAR10(root='./data', train=False, transform=
            test_transform, download=True)
        num_classes = 10

    train_loader = DataLoader(train_dataset, batch_size=batch_size,
        shuffle=True, num_workers=4, pin_memory=True)
    test_loader = DataLoader(test_dataset, batch_size=batch_size,
        shuffle=False, num_workers=4, pin_memory=True)

    if not is_random_baseline and pretrained_path:
        simclr_model = SimCLRNet()
        simclr_model.load_state_dict(torch.load(pretrained_path))
        if use_projector:
            backbone = simclr_model
            in_features = 128
        else:
            backbone = simclr_model.backbone
            in_features = 512
    elif pretrained_sl_path:
        sl_model = nn.Sequential(ModifiedResNet18(), LinearClassifier(
            in_features=512, num_classes=10))
        sl_model.load_state_dict(torch.load(pretrained_sl_path))
        backbone = sl_model[0]
        in_features = 512
    else:
        backbone = ModifiedResNet18()
        in_features = 512

    backbone = backbone.to(device)
    # Freeze backbone
    for param in backbone.parameters():
        param.requires_grad = False

    classifier = LinearClassifier(in_features=in_features, num_classes=
        num_classes).to(device)

```

```

optimizer = optim.Adam(classifier.parameters(), lr=lr, weight_decay
                        =wd)
criterion = nn.CrossEntropyLoss()

logs = {"loss": [], "test_acc": []}

for epoch in range(1, epochs + 1):
    classifier.train()
    total_loss = 0
    pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{epochs}")
    for imgs, targets in pbar:
        imgs, targets = imgs.to(device), targets.to(device)

        with torch.no_grad():
            features = backbone(imgs)
            if isinstance(features, tuple):
                features = features[1] # Use projector output

        optimizer.zero_grad()
        outputs = classifier(features)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    logs["loss"].append(avg_loss)

    # Eval
    classifier.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for imgs, targets in test_loader:
            imgs, targets = imgs.to(device), targets.to(device)
            features = backbone(imgs)
            if isinstance(features, tuple):
                features = features[1]
            outputs = classifier(features)
            _, predicted = outputs.max(1)
            total += targets.size(0)
            correct += predicted.eq(targets).sum().item()

    acc = 100. * correct / total
    logs["test_acc"].append((epoch, acc))
    print(f"Epoch {epoch} | Loss: {avg_loss:.4f} | Test Acc: {acc
        :.2f}%")

```

```

torch.save(classifier.state_dict(), save_name)
with open(save_name.replace(".pth", "_logs.json"), "w") as f:
    json.dump(logs, f)

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--pretrained", type=str, default=None)
    parser.add_argument("--pretrained_sl", type=str, default=None)
    parser.add_argument("--epochs", type=int, default=100)
    parser.add_argument("--random", action="store_true")
    parser.add_argument("--save_name", type=str, default=None)
    parser.add_argument("--dataset", type=str, default="cifar10")
    parser.add_argument("--use_projector", action="store_true")
    args = parser.parse_args()

    if args.random:
        save_name = args.save_name if args.save_name else "
            linprobe_random_baseline.pth"
        train_linprobe(is_random_baseline=True, save_name=save_name,
            epochs=args.epochs, dataset_name=args.dataset)
    elif args.pretrained_sl:
        save_name = args.save_name if args.save_name else "
            linprobe_sl_baseline.pth"
        train_linprobe(pretrained_sl_path=args.pretrained_sl, save_name
            =save_name, epochs=args.epochs, dataset_name=args.dataset)
    else:
        save_name = args.save_name if args.save_name else "
            linprobe_simclr.pth"
        train_linprobe(pretrained_path=args.pretrained, save_name=
            save_name, epochs=args.epochs, dataset_name=args.dataset,
            use_projector=args.use_projector)

```

A.8 plot_results.py

```

import json
import matplotlib.pyplot as plt
import os

def plot_simclr(log_file, ax_loss=None, ax_acc=None, label="SimCLR"):
    if not os.path.exists(log_file): return False
    with open(log_file, "r") as f:
        logs = json.load(f)

    losses = logs["loss"]
    epochs_loss = range(1, len(losses) + 1)

    accs = logs["knn_acc"]
    epochs_acc = [x[0] for x in accs]

```

```

acc_vals = [x[1] for x in accs]

if ax_loss: ax_loss.plot(epochs_loss, losses, label=label)
if ax_acc: ax_acc.plot(epochs_acc, acc_vals, label=label, marker='o')
return True

def plot_supervised(log_file, ax_loss=None, ax_acc=None, label="Supervised"):
    if not os.path.exists(log_file): return False
    with open(log_file, "r") as f:
        logs = json.load(f)

    losses = logs["loss"]
    epochs_loss = range(1, len(losses) + 1)

    accs = logs["test_acc"]
    epochs_acc = [x[0] for x in accs]
    acc_vals = [x[1] for x in accs]

    if ax_loss: ax_loss.plot(epochs_loss, losses, label=label)
    if ax_acc: ax_acc.plot(epochs_acc, acc_vals, label=label)
    return True

def get_final_acc(log_file):
    if not os.path.exists(log_file): return 0.0
    with open(log_file, "r") as f:
        logs = json.load(f)
    return logs["test_acc"][-1][1]

def main():
    # 1. Main loss and kNN curve for SimCLR Baseline
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
    plot_simclr("simclr_baseline_logs.json", ax1, ax2, "Baseline SimCLR
                (bs=256, temp=0.5)")
    ax1.set_title("Training NT-Xent Loss")
    ax1.set_xlabel("Epochs")
    ax1.set_ylabel("Loss")
    ax1.legend()
    ax2.set_title("kNN Monitor Accuracy (k=20)")
    ax2.set_xlabel("Epochs")
    ax2.set_ylabel("Accuracy (%)")
    ax2.legend()
    plt.tight_layout()
    plt.savefig("simclr_learning_curves.png", dpi=300)
    print("Saved simclr_learning_curves.png")

    # 2. Supervised Learning loss and accuracy curve
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

```

```

plot_supervised("sl_baseline_logs.json", ax1, ax2, "Supervised
    ResNet-18")
ax1.set_title("Training Cross-Entropy Loss")
ax1.set_xlabel("Epochs")
ax1.set_ylabel("Loss")
ax1.legend()
ax2.set_title("Test Set Accuracy")
ax2.set_xlabel("Epochs")
ax2.set_ylabel("Accuracy (%)")
ax2.legend()
plt.tight_layout()
plt.savefig("supervised_learning_curves.png", dpi=300)
print("Saved supervised_learning_curves.png")

# 3. Baseline Comparisons Bar Chart (CIFAR-10)
acc_rand = get_final_acc("linprobe_random_baseline_logs.json")
acc_simclr = get_final_acc("linprobe_simclr_logs.json")
acc_sl = get_final_acc("sl_baseline_logs.json")

plt.figure(figsize=(8, 6))
bars = plt.bar(["Random Backbone", "SimCLR SSL", "Supervised"], [
    acc_rand, acc_simclr, acc_sl], color=['gray', 'blue', 'green'])
plt.title("CIFAR-10 Classification Accuracy")
plt.ylabel("Accuracy (%)")
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 1, f'{yval:.2f}
        }%', ha='center', va='bottom')
plt.tight_layout()
plt.savefig("baseline_comparisons.png", dpi=300)
print("Saved baseline_comparisons.png")

# 4. Temperature Ablations Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
plot_simclr("simclr_baseline_logs.json", ax1, ax2, "temp=0.5 (
    Baseline)")
plot_simclr("simclr_temp5_logs.json", ax1, ax2, "temp=5.0")
plot_simclr("simclr_temp01_logs.json", ax1, ax2, "temp=0.1")
ax1.set_title("Loss with Temperature Variations")
ax1.set_xlabel("Epochs")
ax1.set_ylabel("Loss")
ax1.legend()
ax2.set_title("kNN Accuracy with Temperature Variations")
ax2.set_xlabel("Epochs")
ax2.set_ylabel("Accuracy (%)")
ax2.legend()
plt.tight_layout()
plt.savefig("ablation_temperature.png", dpi=300)
print("Saved ablation_temperature.png")

```

```

# 5. Batch Size Ablations Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
plot_simclr("simclr_baseline_logs.json", ax1, ax2, "BS=256 (
    Baseline)")
plot_simclr("simclr_bs128_logs.json", ax1, ax2, "BS=128")
plot_simclr("simclr_bs64_logs.json", ax1, ax2, "BS=64")
plot_simclr("simclr_bs32_logs.json", ax1, ax2, "BS=32")
ax1.set_title("Loss with Batch Size Variations")
ax1.set_xlabel("Epochs")
ax1.set_ylabel("Loss")
ax1.legend()
ax2.set_title("kNN Accuracy with Batch Size Variations")
ax2.set_xlabel("Epochs")
ax2.set_ylabel("Accuracy (%)")
ax2.legend()
plt.tight_layout()
plt.savefig("ablation_batch_size.png", dpi=300)
print("Saved ablation_batch_size.png")

# 6. Projector Ablations Bar Chart
acc_simclr = get_final_acc("linprobe_simclr_logs.json") # Baseline
backbone
acc_noproj = get_final_acc("linprobe_simclr_noproj_logs.json") #
Trained without proj
acc_projout = get_final_acc("linprobe_simclr_projout_logs.json") #
Linprobe on proj outputs

plt.figure(figsize=(10, 6))
bars = plt.bar(["Standard (Encoder)", "Trained w/o Projector", "
    Linear Probe on Projector"],
               [acc_simclr, acc_noproj, acc_projout], color=['blue',
                   , 'orange', 'purple'])
plt.title("Projector Ablations on CIFAR-10")
plt.ylabel("Accuracy (%)")
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 1, f'{yval:.2f
        }%', ha='center', va='bottom')
plt.tight_layout()
plt.savefig("ablation_projector.png", dpi=300)
print("Saved ablation_projector.png")

# 7. Transfer Learning (CIFAR-100) Bar Chart
acc_t_rand = get_final_acc("linprobe_random_cifar100_logs.json")
acc_t_sl = get_final_acc("linprobe_sl_cifar100_logs.json")
acc_t_simclr = get_final_acc("linprobe_simclr_cifar100_logs.json")

plt.figure(figsize=(8, 6))
bars = plt.bar(["Random Backbone", "Supervised (CIFAR-10)", "SimCLR
    (CIFAR-10)"],

```

```

        [acc_t_rand, acc_t_sl, acc_t_simclr], color=['gray',
            'green', 'blue'])
plt.title("Transfer Learning Accuracy on CIFAR-100")
plt.ylabel("Accuracy (%)")
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.5, f'{yval
        :.2f}%', ha='center', va='bottom')
plt.tight_layout()
plt.savefig("transfer_learning_cifar100.png", dpi=300)
print("Saved transfer_learning_cifar100.png")

if __name__ == "__main__":
    main()

```